

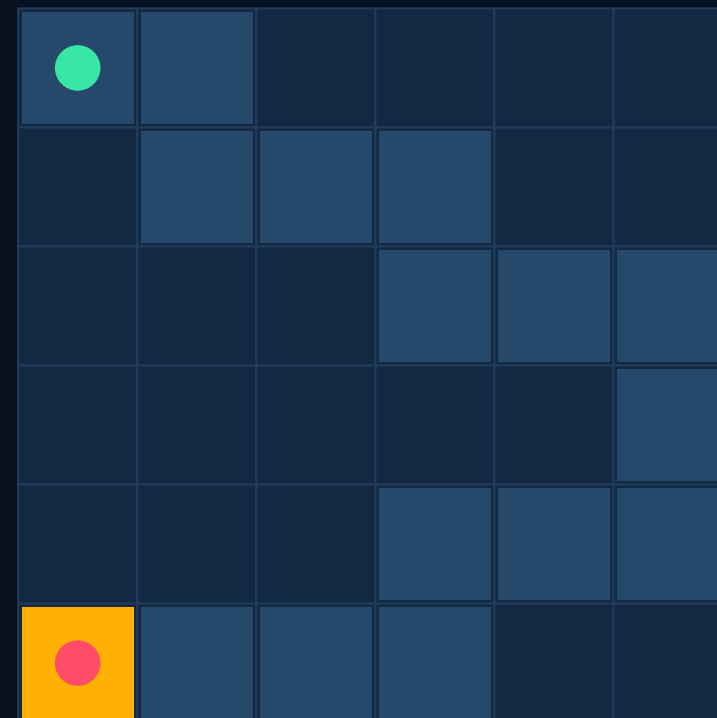
Генерация лабиринта DFS с возвратом

Recursive Backtracking для игрового уровня в Unreal Engine

алгоритм DFS

UE реализация

debug draw

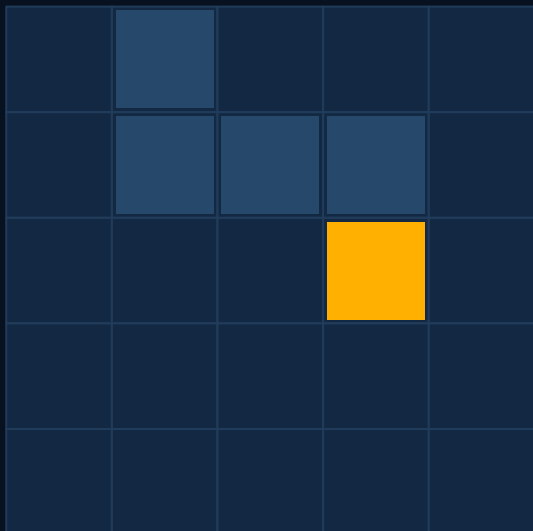


Тупикин Дмитрий

БПИ236

Идея Recursive Backtracking

DFS выбирает случайный путь, а при тупике возвращается назад



1

Выбираем стартовую клетку

2

Ищем непосвященного соседа

3

Удаляем стену и идем дальше

4

Если соседей нет — backtrack

Ключевая особенность: результат — связанный лабиринт.
Каждая клетка достижима из любой другой.

Если клетка стала тупиком, стек возвращает алгоритм к предыдущим развязкам.

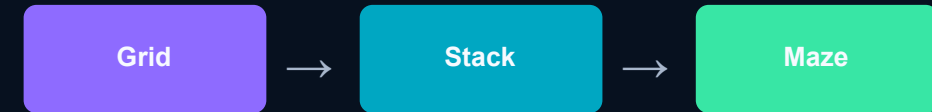
Псевдокод алгоритма

итеративная версия со стеком удобнее для Unreal Engine

```
initGrid(width, height)
start = chooseStartCell()
stack.push(start)
start.visited = true

while stack is not empty:
    current = stack.top()
    neighbors = unvisitedNeighbors(current)

    if neighbors is not empty:
        next = random(neighbors)
        removeWall(current, next)
        next.visited = true
        stack.push(next)
    else:
        stack.pop()
```



- Клетки хранят стены и флаг visited
- Стек хранит путь для возврата
- Случайный выбор соседа делает результат разным
- Seed нужен для повторяемой генерации

Характеристики DFS-лабиринта

почему алгоритм подходит для игрового уровня

Время

$O(W \times H)$

каждая клетка посещается один раз

Память

$O(W \times H)$

массив клеток + стек пути

Результат

**идеальный
лабиринт**

один путь между любыми двумя
клетками

Стиль

**длинные
коридоры**

хорошо для хоррора, квеста, побега

Вывод: алгоритм быстрый, простой и наглядный.

Структуры данных для реализации

минимальный набор, который используется в коде Unreal Engine

FMazeCell

- X, Y
- bVisited
- WallTop / Right / Bottom / Left

TArray<FMazeCell>

- одномерная сетка
- $\text{Index} = Y \times \text{Width} + X$
- быстрый доступ по координатам

Stack + RandomStream

- TArray<int32> Stack
- FRandomStream Seed
- направления FIntPoint

Архитектура Unreal Engine

один Actor-класс: проще подключить, показать и защитить



Компоненты отображения:

- UInstancedStaticMeshComponent для стен
- отдельный Instance Mesh для пола
- маркеры START / EXIT

Debug-демонстрация:

- DrawDebugBox: клетки
- DrawDebugLine: стены
- DrawDebugSphere: старт и выход

Код разделён стандартно для Unreal: MazeGenerator.h содержит объявления, MazeGenerator.cpp — реализацию функций.

Изменяемые параметры

они доказывают, что лабиринт не захардкожен

Width / Height

размер сетки

Cell Size

масштаб уровня

Seed

повторяемость

Start Mode

Random / TopLeft / Center

Draw Debug

отрисовка клеток и стен

Use Random Seed

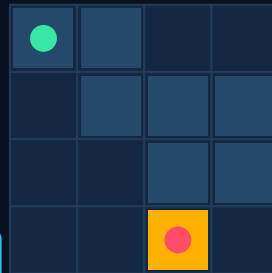
новый лабиринт каждый раз

Extra Connections %

циклы и альтернативные пути

Horizontal Bias

стиль коридоров



На защите достаточно изменить Seed, Width / Height и две модификации, чтобы показать многократную работу проекта.

Seed позволяет получить тот же лабиринт повторно — это удобно для проверки и демонстрации.

Модификации для оценки 9–10

базовый DFS + дополнительные настройки сложности и стиля

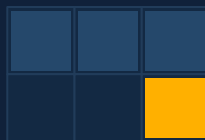
Extra Connections Percent

После генерации часть стен дополнительно удаляется. Появляются циклы и альтернативные маршруты.



Horizontal Bias

Во время DFS меняется предпочтение горизонтальных или вертикальных направлений.



Farthest Exit

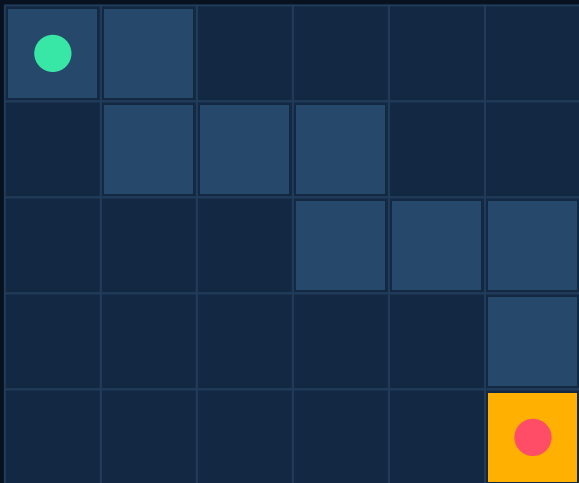
Выход выбирается как самая дальняя достижимая клетка от старта через BFS.



Для максимальной оценки на защите нужно явно показать изменение этих параметров, а не только размер и Seed.

Debug-визуализация и маркеры

что означают линии и шары на уровне



Зелёные / синие клетки
структура сгенерированного лабиринта

Красные линии
стены между клетками

Синий шар
START — стартовая клетка

Жёлтый шар
EXIT — самая дальняя клетка от старта

Маркеры — это не обязательно игровые объекты. Это визуальные подсказки для проверки алгоритма и объяснения на защите.

Как пользоваться генератором

короткая инструкция для проверяющего

- 1 Выделить Actor MazeGenerator на уровне
- 2 В Details изменить Width / Height / Seed
- 3 Назначить Cube в Floor Mesh, Wall Mesh, Marker Mesh
- 4 Включить Draw Debug для проверки структуры
- 5 Изменить Extra Connections % и Horizontal Bias
- 6 Нажать Generate Maze или изменить параметр для пересборки

Если геометрия не видна, значит не назначены Mesh-поля. Для проверки можно использовать стандартный Cube.

Сценарии проверки

что быстро показать проверяющему

Тест 1 — обычный DFS

Width 15 × Height 15
Seed 12345
Extra Connections = 0.0
Horizontal Bias = 0.5

ожидаемо: много тупиков, один маршрут между точками

Тест 2 — дополнительные пути

Width 20 × Height 20
Seed 77777
Extra Connections = 0.15
Horizontal Bias = 0.5

ожидаемо: появляются циклы и обходные маршруты

Тест 3 — стиль коридоров

Width 25 × Height 15
Seed 33333
Extra Connections = 0.1
Horizontal Bias = 0.9

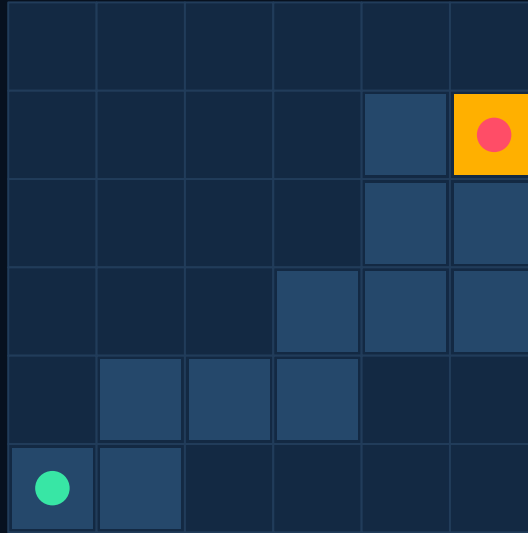
ожидаемо: больше горизонтальных коридоров

Концепт игры: “Побег из живого лабиринта”

DFS создаёт запутанный уровень с длинными коридорами

Игровой цикл

- генерация нового уровня
- поиск ключа
- обход врагов и ловушек
- поиск выхода
- повышение сложности



Почему DFS подходит

- всегда есть путь
- быстро генерируется
- хорошо даёт длинные коридоры
- легко добавить модификации
- удобно проверять через Seed

На текущем этапе проект демонстрирует генератор уровня. START и EXIT уже определяются автоматически.

Итог и источники

что реализовано и на что обратить внимание при защите

- DFS / Recursive Backtracking реализован
- генерация работает при разных Seed и размерах
- есть параметры для демонстрации
- есть Debug Draw средствами Unreal Engine
- добавлены модификации Extra Connections и Horizontal Bias

“Я реализовал базовый DFS с возвратом, добавил параметризацию и две модификации алгоритма. Debug-отрисовка показывает клетки, стены, старт и выход.”

[Jamis Buck — Maze Generation: Recursive Backtracking](#)

[Wikipedia — Maze generation algorithm](#)

[Epic Games — Unreal Engine Debug Drawing Documentation](#)

[Epic Games — Instanced Static Mesh Component Documentation](#)

Материалы задания: Практикум по генерации лабиринтов